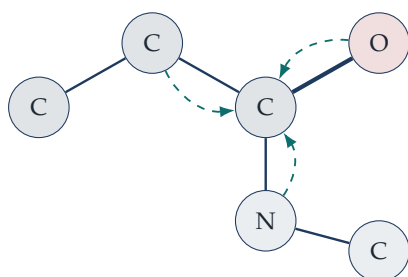


A CLASSROOM TUTORIAL

Graph Neural Networks for Chemists

From the bond table to equivariant message passing



A molecule is a graph, and a graph neural model is a fingerprint whose hash function is learned. The tutorial develops that statement from first principles: how a structure becomes a tensor, why atom ordering must not matter, what one layer of message passing computes, how the published architectures differ, where the method provably fails, and what changes once three-dimensional geometry enters. Worked arithmetic on a four-atom molecule, fifty lines of dependency-free code, and thirteen problems with full solutions are included.

Built on *A Gentle Introduction to Graph Neural Networks* by B. Sanchez-Lengeling, E. Reif, A. Pearce and A. B. Wiltschko, *Distill* **6**, e33 (2021), doi:10.23915/distill.00033. All text, figures, derivations, worked numbers, code and exercises in the present document are original teaching material prepared for classroom use. The Distill article is recommended as the accompanying interactive reading.

How to use this tutorial

The document is written to be read linearly, with a pencil in hand. Four kinds of boxes recur throughout.

Definition

Formal statements. Everything that is later assumed is stated here first.

Chemist's note

The translation between a graph-theoretic statement and the chemistry it encodes.

Worked example

Arithmetic carried out in full on a four-atom molecule. Every number can be reproduced by hand or with the code in Section 15.

Common pitfall

Failure modes that appear in real molecular property prediction and are easy to miss in a first pass.

Prerequisites

Linear algebra at the level of matrix multiplication and eigenvalues, elementary probability, and familiarity with a feed-forward neural model trained by gradient descent. No prior graph theory is assumed. Chemistry background is assumed only in the illustrations, and every chemical example is explained in words.

Learning outcomes

On completing the tutorial and its exercises, a student should be able to:

1. convert a molecule into node, edge and global feature tensors, and justify each featurisation choice;
2. state what permutation invariance and permutation equivariance mean, and verify them for a given operation;
3. write down the message, aggregation, update and readout equations of a message-passing model, and execute one layer by hand;
4. place GCN, GraphSAGE, GIN and GAT within a single unifying formalism, and say what each adds;
5. explain why depth helps up to a point and why it then hurts, in terms of receptive field, oversmoothing and oversquashing;
6. identify pairs of molecules a two-dimensional bond graph model provably cannot distinguish, and name the remedies;
7. design an honest train/test protocol for a molecular dataset, including the choice of split;
8. explain why interatomic potentials require equivariant rather than merely invariant architectures.

Notation

Symbol and meaning

$\mathcal{G} = (\mathcal{V}, \mathcal{E})$	a graph, its nodes and edges	n	number of nodes, $ \mathcal{V} $
i, j	node indices (atoms)	m	number of edges, $ \mathcal{E} $
$\mathcal{N}(i)$	neighbours of node i	d_i	degree of node i
$\mathbf{x}_i \in \mathbb{R}^f$	input feature vector of node i	$\mathbf{X} \in \mathbb{R}^{n \times f}$	node feature matrix
$\mathbf{e}_{ij}$	feature vector of edge (i, j)	$\mathbf{u}$	global (graph-level) feature vector
$\mathbf{h}_i^{(k)}$	hidden state of node i after k layers	$\mathbf{H}^{(k)}$	stacked hidden states
$\mathbf{m}_{j \rightarrow i}$	message sent from j to i	$\oplus$	permutation-invariant aggregator
$\mathbf{A}$	adjacency matrix	$\mathbf{P}$	permutation matrix
ϕ	a learned function, usually a small MLP	σ	elementwise nonlinearity

Contents

How to use this tutorial	2
1 Why a chemist should care about graphs	6
1.1 The representation problem	6
1.2 Where else graphs appear	6
2 Anatomy of a graph	7
2.1 Directed and undirected edges	7
3 From a molecule to tensors	8
3.1 Node features	8
3.2 Two ways to store connectivity	8
3.3 The ordering problem	8
4 Three kinds of prediction task	9
5 A first attempt, and why it fails	10
6 Pooling: moving information between levels	11
7 Message passing	11
7.1 The three steps	11
7.2 Matrix form, and why message passing is a walk	12
7.3 A layer executed by hand	13
8 The architecture zoo, in one formalism	13
8.1 What each addition is for	14
8.2 The global attribute and long-range information	14
9 Depth: how much is enough?	15
9.1 Receptive field	15
9.2 Oversmoothing	16
9.3 Oversquashing	16
10 What message passing can and cannot see	16
10.1 The colour refinement bound	16
10.2 Consequences a chemist will meet	17
10.3 Choosing an aggregator	17
11 Training a molecular model honestly	18
11.1 Batching graphs of different sizes	18
11.2 The split is the experiment	18
11.3 A checklist before believing a number	18
11.4 Empirical regularities	19
12 Interpretation and generation	19
12.1 Explaining a prediction	19
12.2 Generating molecules	19
13 Beyond the bond graph: three dimensions and equivariance	20

14 Laboratory: a message-passing model in fifty lines	21
14.1 What to look at in the output	23
14.2 The same model in a production library	23
14.3 Suggested laboratory sequence	24
15 Problems	24
A Solutions	25
B Glossary	26
C Annotated further reading	26

1 Why a chemist should care about graphs

A chemist already thinks in graphs. A structural formula is a set of atoms together with a statement about which pairs of atoms are bonded, and nothing else. Bond lengths, angles and the embedding in three-dimensional space are deliberately suppressed, yet an enormous amount of chemistry survives the suppression. Functional group reactivity, aromaticity, degrees of unsaturation and most of retrosynthetic logic are properties of the connectivity alone.

The machine learning question is therefore not whether molecules are graphs. The question is how to feed a graph to a differentiable model.

1.1 The representation problem

Classical cheminformatics answers by hand-crafting a fixed-length vector. An extended-connectivity fingerprint enumerates circular atom environments up to a chosen radius, hashes each one into a bit, and returns a sparse binary vector. The procedure works, is fast, and remains a strong baseline. It has three structural weaknesses.

1. *The features are frozen.* A fingerprint is designed before the target property is known. Whatever the model needs that the hash did not preserve is lost irrecoverably.
2. *Hash collisions are uninterpretable.* Two chemically unrelated environments can set the same bit, and no gradient can undo the confusion.
3. *The radius is a fixed hyperparameter.* Long-range effects, such as a substituent influencing a reaction centre five bonds away, need a large radius, which inflates the descriptor and dilutes the local information.

A graph neural model replaces the fixed hash with a learned, differentiable analogue of the same idea. Each atom accumulates information from its environment, layer by layer, and the rule for accumulating is trained against the property of interest. The radius becomes the depth of the model, and the descriptor becomes a byproduct of training rather than an input to it.

Chemist's note: the fingerprint analogy is exact

A message-passing model with K layers and sum aggregation computes, for each atom, a vector summarising its environment out to K bonds. Replace the learned functions with a hash and the sum with a set union, and the extended-connectivity fingerprint is recovered exactly. A GNN is a fingerprint whose hash function is fitted by gradient descent.

1.2 Where else graphs appear

The formalism is not chemistry-specific, and it is worth knowing the neighbours. Reaction databases are graphs whose nodes are molecules and whose edges are transformations. Citation graphs connect papers by reference. Crystal structures become graphs once a neighbour cutoff is imposed. Even an image can be viewed as a graph in which every pixel is a node bonded to its eight neighbours, and a sentence as a chain in which every token points to the next. Images and text are rarely treated that way in practice, because their rigid grid structure permits far more efficient specialised operations, but the exercise is instructive: it shows that a convolution is the special case of message passing that arises when every node has the same number of neighbours in the same geometric arrangement.

The central difficulty in one sentence.

Molecules have no canonical atom ordering, no fixed number of atoms, and no fixed number of bonds per atom, whereas the tensors consumed by neural models have a fixed shape and a definite ordering of their entries.

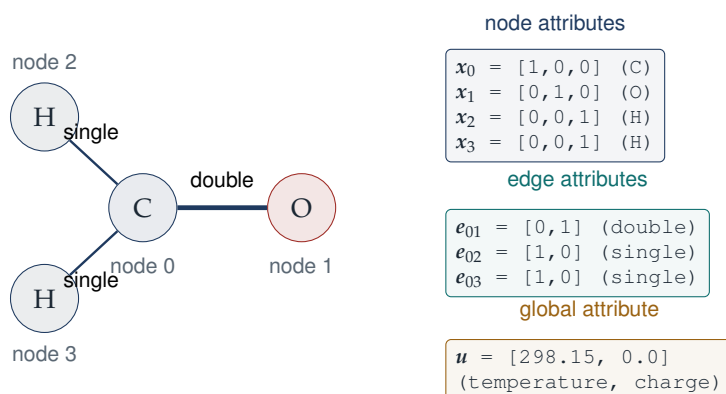


Figure 1: Formaldehyde, CH_2O , as an attributed graph. The atom labels 0 to 3 are arbitrary and carry no chemical meaning, a fact that will shortly become the central design constraint. The same molecule is used for every worked example in this tutorial.

2 Anatomy of a graph

Graph

A **graph** $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ consists of a finite set of **nodes** $\mathcal{V} = \{1, \dots, n\}$ and a set of **edges** $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$. An edge (i, j) is said to connect i and j . The **neighbourhood** of node i is $\mathcal{N}(i) = \{j : (j, i) \in \mathcal{E}\}$ and its **degree** is $d_i = |\mathcal{N}(i)|$.

Three levels of information can be attached to a graph, and a well-designed model uses all three.

Level	Symbol	Chemical content
Node	$x_i \in \mathbb{R}^{f_v}$, one per atom	element, formal charge, hybridisation, aromatic flag, ring membership, number of attached hydrogens
Edge	$e_{ij} \in \mathbb{R}^{f_e}$, one per bond	bond order, conjugation, ring bond flag, stereochemical label
Global	$u \in \mathbb{R}^{f_u}$, one per molecule	temperature, solvent, pH, total charge, spin multiplicity, or a learned summary

2.1 Directed and undirected edges

A covalent bond has no preferred direction, so a molecular graph is undirected. Implementations nevertheless almost always store each undirected bond as two directed edges, $(i \rightarrow j)$ and $(j \rightarrow i)$, because message passing is naturally expressed as a flow along directed edges. A hydrogen bond donor and acceptor relation, a reaction arrow, or a citation is genuinely directed and is stored once.

Common pitfall: forgetting the reverse edges

If a molecular graph is built with one directed edge per bond, information flows in one direction only. Atoms will receive messages from some neighbours and not others, the model will silently depend on the arbitrary order in which the bond block of the file was written, and accuracy will degrade in a way that is very hard to diagnose. Always symmetrise, and check that the number of stored directed edges is exactly twice the number of bonds.

3 From a molecule to tensors

3.1 Node features

Each atom is mapped to a fixed-length vector. A minimal and entirely standard choice concatenates several one-hot blocks:

Block	Encoding	Width
element	one-hot over {C, N, O, F, S, Cl, other}	7
degree	one-hot over {1,2,3,4}	4
formal charge	one-hot over $\{-1, 0, +1\}$	3
hybridisation	one-hot over {sp, sp ² , sp ³ }	3
aromaticity	binary	1
attached hydrogens	one-hot over {0,1,2,3}	4
in ring	binary	1
		$f_v = 23$

Two decisions deserve comment. First, categorical variables are one-hot encoded rather than given integer codes, because an integer code asserts an ordering and a distance that do not exist. Encoding carbon as 6 and nitrogen as 7 tells the model that nitrogen is closer to carbon than oxygen is, which is meaningless. Second, hydrogens may be represented explicitly as nodes or folded into a count on the heavy atom. Implicit hydrogens give smaller graphs and faster training, explicit hydrogens are required whenever hydrogen positions matter, as in hydrogen bonding or in any three-dimensional treatment.

3.2 Two ways to store connectivity

Adjacency matrix

The *adjacency matrix* of a graph on n nodes is $A \in \mathbb{R}^{n \times n}$ with $A_{ij} = 1$ if $(i, j) \in \mathcal{E}$ and $A_{ij} = 0$ otherwise. For an undirected graph A is symmetric.

For formaldehyde with the labelling of Figure 1,

$$A = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}, \quad X = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}. \quad (1)$$

The adjacency matrix is convenient for derivations and catastrophic for storage. Its size grows as n^2 while the number of bonds grows roughly as n , so a protein with 10^4 atoms would require 10^8 entries of which fewer than 10^{-3} are nonzero. The alternative is an *edge list*, two integer arrays of length m holding the source and destination of every directed edge:

$$\text{src} = [0, 1, 0, 2, 0, 3] \quad \text{dst} = [1, 0, 2, 0, 3, 0]$$

Storage is now $O(m)$ rather than $O(n^2)$, and the gather operations required by message passing become simple array indexing. Every production library uses the edge list. The adjacency matrix survives in the literature because it makes the mathematics compact.

3.3 The ordering problem

The atom labels were arbitrary. Relabelling the atoms must not change any predicted property, because the molecule has not changed.

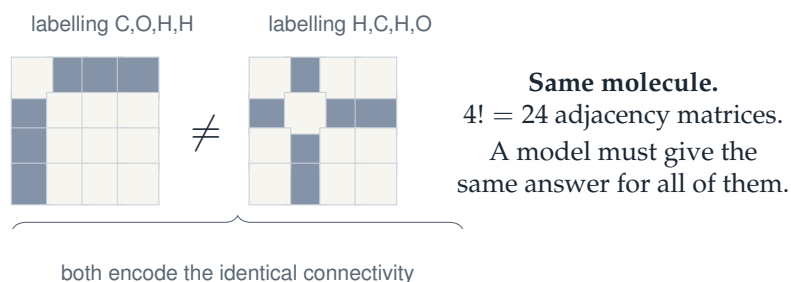


Figure 2: The ordering problem. A permutation of the atom labels produces a different adjacency matrix for the same molecule. Any architecture whose output depends on the labelling is learning an artefact of the file format.

Permutation invariance and equivariance

Let P be an $n \times n$ permutation matrix. A graph-level function f is **permutation invariant** if

$$f(PX, PAP^T) = f(X, A) \quad \text{for every } P. \quad (2)$$

A node-level function F is **permutation equivariant** if

$$F(PX, PAP^T) = PF(X, A) \quad \text{for every } P. \quad (3)$$

Invariance is required of the final prediction of a molecular property. Equivariance is required of every intermediate layer, so that node i keeps its own state whatever its index happens to be.

A graph on n nodes admits $n!$ labellings, and each produces a different adjacency matrix describing the identical molecule. Feeding A to a dense feed-forward model is therefore hopeless: the model would have to learn all $n!$ equivalent presentations of every training example. Convolutional models fail for the same reason, since they assume a spatial ordering that a molecule does not have.

Worked example: two labellings of formaldehyde

Relabel the atoms in the order H, C, H, O, that is $\pi = (2, 0, 3, 1)$ in the sense that new position 0 holds old node 2. The adjacency matrix becomes

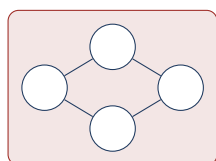
$$A' = PAP^T = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \neq A,$$

yet the molecule is unchanged. Section 14 evaluates a two-layer message-passing model on both labellings and obtains the identical graph-level output $y = 0.492022$, with a difference of exactly zero. The invariance is structural, not approximate, and it holds before any training has taken place.

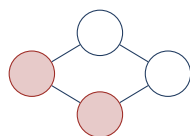
4 Three kinds of prediction task

Everything a GNN is asked to do falls into one of three categories, distinguished only by where the prediction lives.

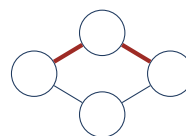
Level	Output shape	Chemical examples
Graph	one vector per molecule	solubility, HOMO–LUMO gap, atomisation energy, toxicity, odour class, binding affinity
Node	one vector per atom	partial charge, NMR chemical shift, atomic force, reactive site identification, pK_a of a specific proton
Edge	one vector per pair	bond order prediction, bond dissociation energy, existence of an interaction in a protein–ligand complex, reaction bond changes



graph level
one label for
the whole molecule



node level
one label per atom



edge level
one label per bond
or per atom pair

Figure 3: The three prediction levels. A single architecture serves all three. Only the final readout changes.

Chemist’s note: choose the level honestly

A common error is to force a node-level problem into a graph-level model. Predicting whether a molecule contains a reactive site is a graph-level task and is easy. Predicting *which* atom is the reactive site is a node-level task, requires per-atom labels, and is far more useful synthetically. The second is not obtainable from the first by inspection of a graph-level model, and attempts to extract it by attribution are approximate at best.

Exercise 1. A molecule has n atoms and m bonds. Write the memory cost of the adjacency matrix and of the edge list, in units of stored numbers, for benzene with explicit hydrogens ($n = 12$, $m = 12$) and for a small protein ($n = 1500$, $m = 1530$). At what value of n does the edge list become the cheaper option, assuming $m \approx 1.05 n$?

Exercise 2. Give a chemical property that is invariant under atom relabelling, one that is equivariant, and one that is neither. Justify each in one sentence.

Exercise 3. Explain why one-hot encoding of the element is preferable to using the atomic number directly as a single feature. Then give one situation in which the atomic number as a scalar feature is in fact the better choice.

5 A first attempt, and why it fails

Build the simplest thing that respects the ordering constraint. Apply the same small multilayer perceptron independently to every atom feature vector, apply another to every bond feature vector, and a third to the global vector:

$$h_i^{(k+1)} = \phi_v(h_i^{(k)}), \quad e_{ij}^{(k+1)} = \phi_e(e_{ij}^{(k)}), \quad u^{(k+1)} = \phi_u(u^{(k)}). \quad (4)$$

The parameters are shared across atoms, which is exactly what makes the layer equivariant: permuting the atoms permutes the outputs and nothing else. Stacking such layers and finishing with a sum over atoms gives a legitimate, permutation-invariant molecular property predictor.

The model is also nearly useless. Equation (4) never consults A . Connectivity enters only in the final sum, so the model sees a molecule as an unordered bag of atoms and an unordered bag of bonds. Ethanol and dimethyl ether, which share the formula C_2H_6O and the same multiset of bond types, are indistinguishable to it. So are all constitutional isomers of a given bond composition. Whatever the model achieves, it achieves by counting.

Diagnosis.

Sharing parameters across atoms buys equivariance. Reading the neighbourhood buys chemistry. The remainder of the tutorial is about the second.

6 Pooling: moving information between levels

Before adding connectivity to the interior of the model, note that the three attribute levels frequently mismatch the task. A dataset may carry bond features and no atom features, while the label is per atom. The remedy is *pooling*, which is nothing more than gather-then-aggregate.

Pooling

Let $\rho_{\mathcal{E} \rightarrow \mathcal{V}}$ denote pooling from edges to nodes. For each node i , gather the feature vectors of all incident edges and aggregate them with a permutation-invariant operator $\oplus$:

$$\rho_{\mathcal{E} \rightarrow \mathcal{V}}(i) = \bigoplus_{j \in \mathcal{N}(i)} e_{ij}. \quad (5)$$

Pooling from nodes to the graph, $\rho_{\mathcal{V} \rightarrow u}$, aggregates over all nodes and is usually called the *readout*. The aggregator must be invariant to the order of its arguments, which restricts it to operations such as sum, mean, maximum and variance.

Readout by sum is the default for extensive properties and readout by mean for intensive ones, a distinction that is thermodynamically obvious to a chemist and routinely ignored in practice.

Chemist's note: match the readout to the property

Atomisation energy, molecular mass and total dipole magnitude scale with the number of atoms and demand *sum* pooling. A model with mean pooling must learn to multiply by n , which it cannot do, and its errors will grow systematically with molecular size. Conversely, density, average bond length and a normalised score are intensive and are better served by *mean* pooling. Checking the extensivity of the target before choosing the readout is the single cheapest accuracy improvement available.

7 Message passing

7.1 The three steps

Connectivity is brought inside the layer by letting neighbours exchange information. One layer proceeds in three steps.

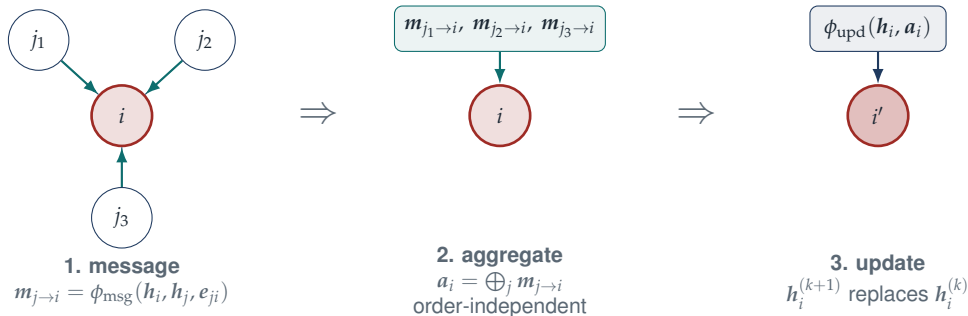


Figure 4: One layer of message passing, viewed from a single atom. The three neighbours contribute independently, their contributions are combined by an order-independent reduction, and the result is merged with the atom’s own state. Every atom in the molecule undergoes the same procedure in parallel, with the same parameters.

A message-passing layer

For every node i with hidden state $h_i^{(k)}$:

$$\text{message} \quad m_{j \rightarrow i}^{(k)} = \phi_{\text{msg}}(h_i^{(k)}, h_j^{(k)}, e_{ji}), \quad j \in \mathcal{N}(i), \quad (6)$$

$$\text{aggregate} \quad a_i^{(k)} = \bigoplus_{j \in \mathcal{N}(i)} m_{j \rightarrow i}^{(k)}, \quad (7)$$

$$\text{update} \quad h_i^{(k+1)} = \phi_{\text{upd}}(h_i^{(k)}, a_i^{(k)}). \quad (8)$$

The learned functions ϕ_{msg} and ϕ_{upd} are shared by every node, and $\bigoplus$ is any permutation-invariant reduction. After K layers a readout produces the prediction:

$$\hat{y} = \psi\left(\bigoplus_{i \in \mathcal{V}} h_i^{(K)}\right). \quad (9)$$

Two properties follow immediately and should be verified by the reader. Equations (6) to (8) are permutation equivariant, because $\bigoplus$ discards the order of the neighbours and the same ϕ is applied everywhere. Equation (9) is permutation invariant, for the same reason. The graph itself is never modified: a message-passing layer takes a graph in and returns a graph out, with identical connectivity and updated attributes.

7.2 Matrix form, and why message passing is a walk

For the simplest choice, $\phi_{\text{msg}}(h_i, h_j, e) = h_j$ with sum aggregation and a linear update, the entire layer collapses to

$$H^{(k+1)} = \sigma(AH^{(k)}W). \quad (10)$$

Reading the product AH entrywise,

$$(AH)_{i\ell} = \sum_j A_{ij}H_{j\ell} = \sum_{j \in \mathcal{N}(i)} H_{j\ell}, \quad (11)$$

which is precisely gather-and-sum over neighbours. Multiplication by the adjacency matrix is one step of message passing.

The observation has a useful corollary. Consider powers of A . The entry $(A^2)_{ij} = \sum_\ell A_{i\ell}A_{\ell j}$ counts walks of length two from i to j , since a term contributes only when both $i-\ell$ and $\ell-j$ are bonds. By induction, $(A^K)_{ij}$ counts walks of length K . Stacking K message-passing layers therefore lets atom i see everything within K bonds, a quantity called the *receptive field*.



Figure 5: Growth of the receptive field along a linear chain. After K layers the central atom has been influenced by every atom within K bonds. The depth of the model is the chemical radius it can perceive.

7.3 A layer executed by hand

Worked example: one message-passing layer on formaldehyde

Take the graph of Figure 1, hidden width $d = 4$, a message function $\phi_{\text{msg}} = \text{ReLU}(\mathbf{W}_m[\mathbf{h}_i \parallel \mathbf{h}_j \parallel \mathbf{e}_{ji}])$, sum aggregation, and an update $\phi_{\text{upd}} = \text{ReLU}(\mathbf{W}_u[\mathbf{h}_i \parallel \mathbf{a}_i])$, with weights drawn once from $\mathcal{N}(0, 0.5^2)$ and never trained. The embedding layer $\mathbf{H}^{(0)} = \text{ReLU}(\mathbf{X}\mathbf{W}_{\text{in}})$ gives

$$\mathbf{H}^{(0)} = \begin{pmatrix} 0.063 & 0.000 & 0.320 & 0.052 \\ 0.000 & 0.181 & 0.652 & 0.474 \\ 0.000 & 0.000 & 0.000 & 0.021 \\ 0.000 & 0.000 & 0.000 & 0.021 \end{pmatrix} \begin{matrix} \text{C} \\ \text{O} \\ \text{H} \\ \text{H} \end{matrix}$$

Six directed messages are computed, one per stored edge. Summing those that arrive at each node gives

$$\mathbf{a} = \begin{pmatrix} 0.872 & 1.123 & 1.701 & 2.447 \\ 0.305 & 1.237 & 0.607 & 0.109 \\ 0.049 & 0.073 & 0.578 & 0.997 \\ 0.049 & 0.073 & 0.578 & 0.997 \end{pmatrix}, \quad \mathbf{H}^{(1)} = \begin{pmatrix} 0.446 & 0.000 & 0.905 & 0.000 \\ 0.502 & 0.000 & 1.964 & 0.000 \\ 0.000 & 0.000 & 0.000 & 0.017 \\ 0.000 & 0.000 & 0.000 & 0.017 \end{pmatrix}.$$

Observe the carbon row of $\mathbf{a}$. Its entries are the largest in the matrix, because carbon has three neighbours while every other atom has one. Sum aggregation encodes degree, and degree is chemistry.

A second layer gives $\mathbf{H}^{(2)}$ with rows (0.529, 0, 1.664, 0) for C, (0.187, 0, 2.288, 0) for O and (0.255, 0, 0.219, 0) for both hydrogens, and sum readout with a linear head returns $\hat{y} = 0.4920$.

Two lessons. The oxygen and the carbon have become distinguishable although the hidden width is only four, so a small model already separates chemical environments. The two hydrogens have *identical* vectors at every layer, and always will, because no sequence of message-passing steps can distinguish atoms that are related by a symmetry of the graph. Section 10 makes that limitation precise.

8 The architecture zoo, in one formalism

Most published architectures are particular choices of ϕ_{msg} , $\oplus$ and ϕ_{upd} . The table below is worth memorising, because it reduces a large literature to a small number of decisions.

Model	Node update	Adds
GCN	$h'_i = \sigma\left(W \sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{d_i d_j}} h_j\right)$	degree normalisation
GraphSAGE	$h'_i = \sigma\left(W_1 h_i + W_2 \text{mean}_{j \in \mathcal{N}(i)} h_j\right)$	separate self weight
GIN	$h'_i = \text{MLP}\left((1 + \epsilon)h_i + \sum_{j \in \mathcal{N}(i)} h_j\right)$	maximal discriminative power
GAT	$h'_i = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} W h_j\right),$ $\alpha_{ij} = \text{softmax}_j\left(a^T [W h_i \ W h_j]\right)$	learned neighbour weights
MPNN with edges	$h'_i = \text{GRU}\left(h_i, \sum_{j \in \mathcal{N}(i)} W(e_{ji}) h_j\right)$	bond-conditioned weights

8.1 What each addition is for

Degree normalisation. Without it, a highly connected atom accumulates a much larger aggregate than a terminal one, and activations grow with each layer. The factor $1/\sqrt{d_i d_j}$ stabilises training at the cost of discarding degree information, which as noted above is chemically meaningful. The trade is real and is the reason GCN underperforms on molecules relative to graphs of citations or of social contacts.

A separate self weight. In GCN the atom’s own state is one summand among many. Giving it its own matrix lets the model weigh what an atom is against what surrounds it, which matters when the element identity is more predictive than the environment.

Attention. Not all neighbours are equally relevant. A scalar score α_{ij} , computed from the pair and normalised across the neighbourhood by a softmax, lets the model concentrate on the informative bond. Permutation invariance is preserved because the score depends only on the pair and the normalisation is over an unordered set. The learned weights are also the closest thing a GNN offers to a free interpretability signal.

Bond-conditioned weights. Making the transformation applied to a message depend on the bond type is the most chemically natural extension of all. A message travelling along an aromatic bond should not be processed like one travelling along a single bond.

Chemist’s note: transformers are message passing in disguise

A transformer computes attention between all pairs of tokens. Interpreted as a graph model, a transformer is a graph attention model on a fully connected graph, in which every node is a neighbour of every other. The distinction between the two families is therefore not the mechanism but the assumed connectivity, sparse and chemically motivated in one case, complete and learned in the other. Graph transformers, which add structural encodings to a complete graph, sit deliberately between the extremes.

8.2 The global attribute and long-range information

After K layers, atoms more than K bonds apart have never communicated. Para substituent effects in a benzene ring require $K \geq 3$, and a substituent at the end of a decyl chain requires $K \geq 10$. Simply increasing K is not viable, for reasons given in Section 9.

Two devices help. A *virtual node* is added to the graph and bonded to every atom, so that any pair of atoms communicates in two hops through it. A *global attribute* u is maintained

alongside the atoms and updated each layer from the pooled atom states, then broadcast back:

$$\mathbf{u}^{(k+1)} = \phi_u \left(\mathbf{u}^{(k)}, \bigoplus_i \mathbf{h}_i^{(k)}, \bigoplus_{ij} \mathbf{e}_{ij}^{(k)} \right), \quad (12)$$

$$\mathbf{h}_i^{(k+1)} = \phi_v \left(\mathbf{h}_i^{(k)}, \mathbf{a}_i^{(k)}, \mathbf{u}^{(k)} \right). \quad (13)$$

The two devices are close relatives, and both trade a small amount of locality for a global communication channel. Both also introduce a risk: a virtual node bonded to everything can wash out local structure if the model comes to rely on it, and the effect is worth monitoring.

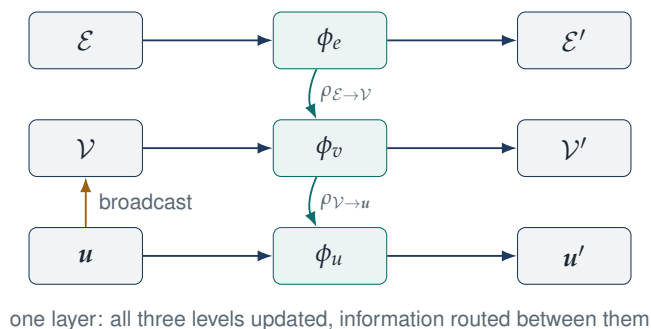


Figure 6: A full layer in the graph-attribute formalism. Edge states are updated first, pooled to the nodes, nodes updated, pooled to the global state, and the global state broadcast back. Restricting attention to the middle row recovers the familiar node-only picture.

Exercise 4. Show that Equation (10) with A replaced by $\tilde{A} = A + I$ is equivalent to a message-passing layer in which the node includes itself in its own neighbourhood. What chemical information is lost if the self-loop is omitted?

Exercise 5. Starting from the GCN update, show that for a d -regular graph the normalisation reduces to a constant factor $1/d$. Name a molecular graph that is regular and one that is very far from regular.

Exercise 6. Write a message function that treats a bond as directed in the sense of electronegativity, so that the message from O to C differs from the message from C to O even when the bond features are symmetric. Is the resulting layer still permutation equivariant?

9 Depth: how much is enough?

Deeper is better in vision and in language modelling. In graph learning the relation is not monotonic, and molecular models are typically two to five layers deep. Three distinct effects compete.

9.1 Receptive field

Depth buys chemical radius, as Figure 5 showed. Predicting the pK_a of a carboxylic acid needs at most two or three bonds of context. Predicting the effect of a remote electron-withdrawing group needs more. Choosing K smaller than the true interaction range guarantees an irreducible error, and no amount of width compensates.

9.2 Oversmoothing

Consider the linear message-passing map $\mathbf{H}^{(k+1)} = \hat{\mathbf{A}}\mathbf{H}^{(k)}$ with $\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1}\tilde{\mathbf{A}}$, the row-normalised adjacency with self-loops. The matrix $\hat{\mathbf{A}}$ is row-stochastic, so its spectral radius is one and repeated application converges to the projection onto the leading eigenvector,

$$\lim_{k \rightarrow \infty} \hat{\mathbf{A}}^k \mathbf{H}^{(0)} = \mathbf{1} \boldsymbol{\pi}^\top \mathbf{H}^{(0)}, \quad (14)$$

where $\boldsymbol{\pi}$ is the stationary distribution of a random walk on the graph. Every row of the limit is identical. All atoms converge to the same vector, and the chemistry is erased.

Oversmoothing

Repeated neighbourhood averaging drives node representations towards a graph-dependent constant. The rate is governed by the spectral gap of $\hat{\mathbf{A}}$, so densely connected graphs smooth fastest. Nonlinearities and residual connections slow the process but do not eliminate it.

Practical countermeasures are residual connections $\mathbf{h}_i^{(k+1)} \leftarrow \mathbf{h}_i^{(k)} + \phi(\cdot)$, concatenation of the states from all layers before readout, layer normalisation, and simply not using twelve layers.

9.3 Oversquashing

A subtler failure appears in the opposite regime. Information from an exponentially growing neighbourhood must be compressed into a fixed-width vector. For a molecule with a bottleneck, such as a single bond joining two ring systems, every message between the two halves is forced through one edge. The representation capacity of the bottleneck is d numbers, while the information attempting to cross grows with the size of the halves.

Symptom	Cause	Remedy
accuracy plateaus with more layers	receptive field already sufficient	stop adding layers
accuracy <i>degrades</i> with more layers	oversmoothing	residuals, jumping connections
long-range effects never learned	oversquashing	virtual node, graph rewiring, wider d
training loss falls, validation does not	memorisation of small data	scaffold split, augmentation

Common pitfall: reading the depth curve backwards

Depth is often tuned on a validation set and the best value reported without diagnosis. Two layers winning over six may mean the property is local, or it may mean the model is oversmoothing. The two are distinguished by adding residual connections and re-running: if six layers now match or beat two, the original failure was oversmoothing and the property was not local at all.

10 What message passing can and cannot see

10.1 The colour refinement bound

The expressive power of standard message passing is known exactly, and the answer is a classical graph algorithm from 1968.

Weisfeiler–Leman colour refinement

Give every node an initial colour equal to its label. Repeat: replace each node’s colour by a hash of the pair consisting of its own colour and the multiset of its neighbours’ colours. Two graphs are declared possibly isomorphic if the multisets of colours agree at every round. The test is called 1-WL.

The correspondence with Equations (6) to (8) is immediate. Colour refinement is message passing with a hash for ϕ and a multiset for $\oplus$.

Theorem (informal).

A message-passing GNN cannot distinguish two graphs that 1-WL cannot distinguish. Architectures whose aggregator and update are injective on multisets, of which GIN is the standard example, attain the bound and are no weaker.

10.2 Consequences a chemist will meet

Cycle counting. 1-WL cannot count cycles of length six. The standard demonstration is the pair consisting of two disjoint benzene-sized rings on one hand and a single twelve-membered ring on the other. Every atom in both has exactly two neighbours of the same type, so all colours are identical forever, and no message-passing model with those inputs can separate a bicyclic system from a macrocycle. The chemical consequence is direct, since aromaticity, ring strain and macrocyclic binding all depend on ring size.

Automorphic atoms. The two hydrogens of formaldehyde received identical embeddings in the worked example, at every layer and for every choice of weights. The behaviour is correct, because the two hydrogens are chemically equivalent. The same mechanism, however, makes enantiomers indistinguishable: a stereocentre and its mirror image have identical bond graphs. Chirality must be injected explicitly, either as a node feature carrying the R/S label or by an aggregator built to be sensitive to the ordering of substituents around a tetrahedral centre.

Remedies. Ring size and other substructure counts can be supplied as extra node or global features, which is cheap and effective. Higher-order architectures that pass messages between tuples of nodes exceed 1-WL at considerable cost. Adding three-dimensional information sidesteps the problem entirely, as Section 13 describes.

10.3 Choosing an aggregator

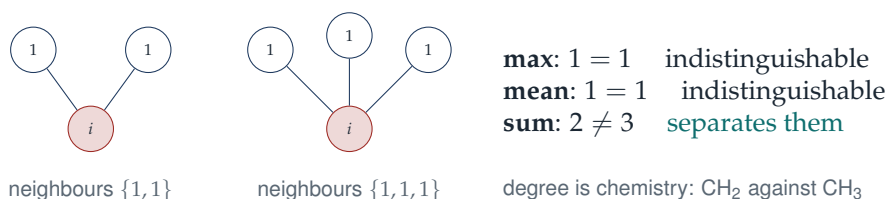


Figure 7: Why sum is the default for molecules. Mean and maximum are blind to the number of identical neighbours, and the number of identical neighbours is exactly what distinguishes a methylene from a methyl group.

No aggregator dominates in every setting. Sum is injective on multisets of bounded size and is the choice that attains the 1-WL bound, at the price of activations that grow with degree. Mean normalises away degree and is preferable when node degree is an artefact of data collection rather than a property of the system, which is common in graphs of citations and rare in molecules. Maximum highlights a single dominant neighbour and is useful when one substituent controls the chemistry. Concatenating several aggregators, with a degree-dependent scaling, is a robust default when the choice is unclear.

11 Training a molecular model honestly

11.1 Batching graphs of different sizes

Molecules have different numbers of atoms, so the tensors cannot be stacked along a batch axis. The standard solution is to treat a batch as one large disconnected graph. Node features are concatenated, edge indices are offset by the running node count, and a `batch` vector records which molecule each atom belongs to. The adjacency matrix of the batch is block diagonal, so no message ever crosses between molecules, and readout is a segment sum over the `batch` vector.

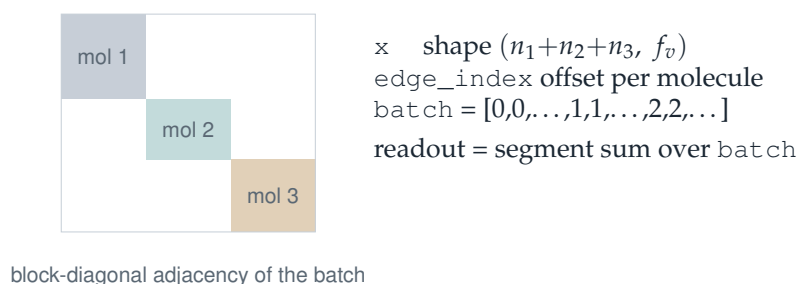


Figure 8: Mini-batching by disjoint union. Three molecules become one graph with three connected components, which a message-passing layer processes in a single call without any padding or masking.

11.2 The split is the experiment

Common pitfall: the random split

A random train/test split of a molecular dataset places close analogues on both sides. The model interpolates within chemical series it has already seen, the reported error is small, and performance on a genuinely new scaffold is far worse. Published improvements on random splits frequently vanish under a scaffold split.

Three protocols, in increasing order of severity:

- **Random split.** Answers the question “how well does the model interpolate within this dataset?” Appropriate only when the deployment distribution matches the training distribution exactly.
- **Scaffold split.** Groups molecules by Bemis–Murcko framework and assigns whole groups to one side. Answers “how well does the model generalise to a new chemical series?” Appropriate for most discovery settings.
- **Time split.** Trains on everything measured before a date and tests on everything after. The most faithful simulation of prospective use, and the most punishing.

11.3 A checklist before believing a number

1. Is the target extensive or intensive, and does the readout match?
2. Was the split random, and if so, why?
3. Is there a baseline? A gradient-boosted model on fingerprints beats a badly tuned GNN more often than the literature admits, and on small datasets it beats a well-tuned one.
4. Are the errors reported with a spread over several random seeds? Graph models on datasets of a few thousand molecules have seed-to-seed variation comparable to the differences between published architectures.

5. Are duplicate or near-duplicate structures present across the split?
6. Does the error grow with molecular size? A systematic trend indicates a readout or normalisation problem rather than a chemical one.

11.4 Empirical regularities

Studies that sweep the design space of graph models on molecular property tasks report a consistent set of tendencies. Parameter count correlates only weakly with accuracy, and competitive models are found at a few thousand parameters. Increasing the hidden width raises mean performance while leaving the best achievable performance largely unchanged. Mean accuracy improves with depth up to a small number of layers, after which the lower tail degrades. Sum aggregation carries a slight advantage on average, without dominating. The clearest and most reliable trend concerns communication: the more attribute levels exchange information, the better the average model, and models with no message passing at all sit firmly at the bottom.

The practical reading is that architectural refinement yields less than featurisation and graph construction. Adding a well-chosen node feature, a sensible global attribute, or an edge that encodes a through-space contact will usually outperform a change of aggregator.

12 Interpretation and generation

12.1 Explaining a prediction

Three families of method are in use. *Attribution* assigns an importance score to each atom and bond by gradient or perturbation analysis, giving a heat map over the structure. *Subgraph extraction* searches for the smallest substructure whose presence preserves the prediction, which returns an object a chemist can read as a functional group. *Attention inspection* reads the learned α_{ij} directly, which is free but should be treated as a hypothesis rather than an explanation, since attention weights are not causal.

Common pitfall: explaining a model that learned the wrong thing

Attribution maps are frequently produced without first checking that the model generalises. A model that has latched onto a dataset artefact, such as the presence of a bromine that correlates with the assay used, will produce a confident and entirely misleading attribution. Explanation is a diagnostic step that follows validation, never a substitute for it.

12.2 Generating molecules

Running the mapping backwards, from a desired property to a structure, is inverse design. Two families dominate. *One-shot* approaches predict an entire adjacency matrix at once, usually within an autoencoder, and face the difficulty that the object being predicted has n^2 entries and no canonical ordering. *Sequential* approaches build the graph by a series of discrete actions, adding an atom or a bond at each step, and are trained by reinforcement learning or as autoregressive models. String representations such as SELFIES sidestep the graph entirely by guaranteeing that every generated string decodes to a valid molecule. Validity, uniqueness and novelty are the three metrics reported, and all three are easy to satisfy simultaneously by a model that is nevertheless chemically useless, so synthetic accessibility and a docking or property filter are essential companions.

13 Beyond the bond graph: three dimensions and equivariance

Everything so far used connectivity only. For thermochemistry, spectroscopy, conformational analysis and anything involving forces, geometry is indispensable. A conformer is defined by coordinates $\{r_i\}$, and the graph is built by a distance cutoff rather than by a bond table.

The symmetry group changes accordingly. Permutation invariance is still required, and to it are added translation, rotation and, for achiral properties, reflection. Energy is invariant under all of them. Forces, dipole moments and polarisability tensors are not: rotating a molecule rotates its forces.

Invariance and equivariance under rotation

Let $R \in SO(3)$. A scalar prediction E is *invariant* if $E(Rr_1, \dots, Rr_n) = E(r_1, \dots, r_n)$. A vector prediction F_i is *equivariant* if $F_i(Rr_1, \dots) = R F_i(r_1, \dots)$. A model whose internal states are all scalars can produce invariant outputs correctly, and can produce equivariant outputs only through the gradient $F_i = -\partial E / \partial r_i$.

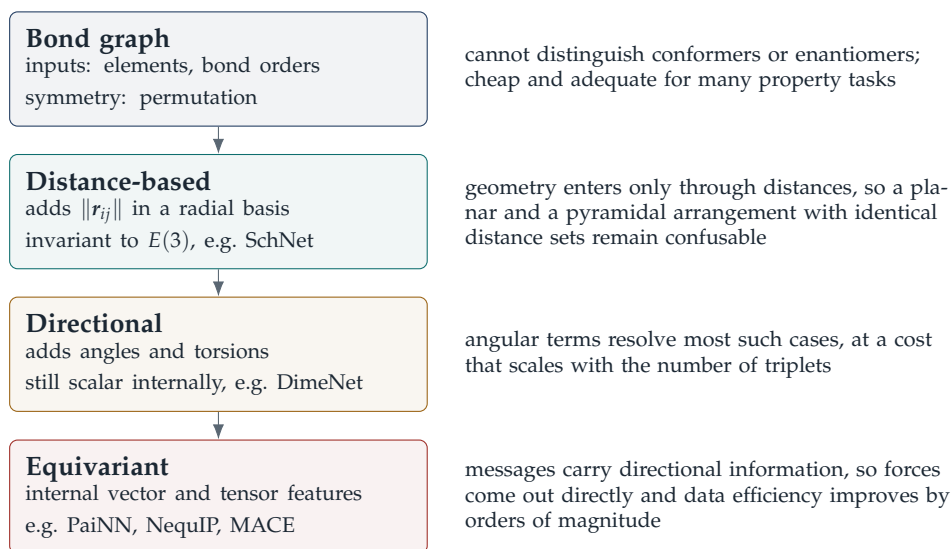


Figure 9: The ladder from a bond table to a machine-learned interatomic potential. Each rung adds geometric information and a corresponding symmetry requirement.

The step that matters most is the last. In an invariant architecture every hidden feature is a scalar, so directional information is destroyed at the first layer and must be rebuilt from angles at every subsequent one. In an equivariant architecture the hidden features are labelled by irreducible representations of the rotation group, and a message carries direction as a vector or higher tensor that rotates with the molecule. Preserving direction throughout is the reason equivariant potentials reach chemical accuracy on training sets one or two orders of magnitude smaller than their invariant predecessors require.

Chemist's note: the bond graph has not been retired

Choosing between the rungs is a question about the target, not about sophistication. A property determined by constitution, such as a solubility class or a synthetic route label, is served well by the bond graph and is only slowed by geometry. A property determined by a potential energy surface, such as a conformer energy, a vibrational frequency or a reaction barrier, cannot be obtained from connectivity at all. Selecting the lowest rung that contains the physics of the target remains the correct discipline.

Exercise 7. Two disjoint six-membered rings and one twelve-membered ring, all carbon, all

with two neighbours per atom. Run two rounds of colour refinement by hand and confirm that the colour multisets are identical. Then propose a single additional node feature that separates them.

Exercise 8. A model predicts atomisation energy with mean pooling. Sketch the expected form of the signed error as a function of the number of heavy atoms, and explain the shape.

Exercise 9. Explain why a model whose hidden features are all scalars can still predict forces correctly, and state the computational price of the mechanism it must use.

Exercise 10. You are given 400 measured solubilities. Argue for or against training a five-layer message-passing model, and name the baseline you would insist on.

14 Laboratory: a message-passing model in fifty lines

The listing below implements everything in Section 7 with no dependencies beyond NumPy. Forward pass only, because the purpose is to make every tensor visible rather than to fit anything. Running it reproduces the numbers quoted in the worked example of Section 7 and verifies permutation invariance to machine precision.

```

1  """
2  mpnn_numpy.py
3  A message-passing graph neural network written from scratch in NumPy.
4  Forward pass only. The point is to make every tensor visible.
5
6  Test molecule: formaldehyde, CH2O.
7  Atom order (arbitrary): 0=C, 1=O, 2=H, 3=H
8  Bonds: 0=1 (double), 0=2 (single), 0=3 (single)
9  """
10
11 import numpy as np
12
13 np.set_printoptions(precision=3, suppress=True)
14 rng = np.random.default_rng(0)
15
16 # -----
17 # 1. Featurisation
18 # -----
19 ATOMS = ["C", "O", "H"]          # one-hot alphabet for nodes
20 BONDS = ["single", "double"]     # one-hot alphabet for edges
21
22 def onehot(x, alphabet):
23     v = np.zeros(len(alphabet))
24     v[alphabet.index(x)] = 1.0
25     return v
26
27 symbols = ["C", "O", "H", "H"]
28 X = np.stack([onehot(s, ATOMS) for s in symbols])          # (4, 3)
29
30 # undirected bonds stored as two directed edges each
31 bonds = [(0, 1, "double"), (0, 2, "single"), (0, 3, "single")]
32 src, dst, etypes = [], [], []
33 for i, j, b in bonds:
34     src += [i, j]
35     dst += [j, i]
36     etypes += [b, b]
37 E = np.stack([onehot(b, BONDS) for b in etypes])          # (6, 2)
38 src = np.array(src)
39 dst = np.array(dst)
40

```

```

41 # dense adjacency, for comparison only
42 A = np.zeros((4, 4))
43 A[src, dst] = 1.0
44
45 # -----
46 # 2. Parameters (random, untrained)
47 # -----
48 d = 4 # hidden width
49 W_in = rng.normal(0, 0.5, (len(ATOMS), d))
50 W_msg = rng.normal(0, 0.5, (2 * d + len(BONDS), d))
51 W_upd = rng.normal(0, 0.5, (2 * d, d))
52 w_out = rng.normal(0, 0.5, (d, 1))
53
54 relu = lambda z: np.maximum(z, 0.0)
55
56 # -----
57 # 3. One message-passing layer
58 # -----
59 def mp_layer(H, E, src, dst, verbose=False):
60     # (a) MESSAGE: one message per directed edge
61     M = relu(np.concatenate([H[src], H[dst], E], axis=1) @ W_msg) # (n_edges,
62     # (b) AGGREGATE: sum messages arriving at each node
63     Agg = np.zeros_like(H)
64     np.add.at(Agg, dst, M) # (n_nodes,
65     # (c) UPDATE
66     H_new = relu(np.concatenate([H, Agg], axis=1) @ W_upd)
67     if verbose:
68         print("messages M =\n", M)
69         print("aggregated Agg =\n", Agg)
70     return H_new
71
72 def readout(H):
73     return float((np.sum(H, axis=0) @ w_out)[0]) # sum pooling + linear head
74
75 def forward(X, E, src, dst, n_layers=2, verbose=False):
76     H = relu(X @ W_in)
77     if verbose:
78         print("H^(0) =\n", H)
79     for k in range(n_layers):
80         H = mp_layer(H, E, src, dst, verbose=verbose)
81         if verbose:
82             print(f"H^({k+1}) =\n", H)
83     return H, readout(H)
84
85 print("=" * 62)
86 print("Adjacency matrix A (atom order C,O,H,H):\n", A)
87 print("Edge list src ->", src, "\n          dst ->", dst)
88 print("=" * 62)
89 H_final, y = forward(X, E, src, dst, n_layers=2, verbose=True)
90 print("graph-level output y =", round(y, 4))
91
92 # -----
93 # 4. Permutation test: relabel the atoms and re-run
94 # -----
95 perm = np.array([2, 0, 3, 1]) # new order: H, C, H, O
96 inv = np.argsort(perm)
97 Xp = X[perm]
98 srcp, dstp = inv[src], inv[dst]
99 Hp, yp = forward(Xp, E, srcp, dstp, n_layers=2)
100
101 print("=" * 62)
102 print("after permutation, node states are the same set, reordered:",

```

```

103     np.allclose(Hp[inv], H_final))
104     print("graph output before / after permutation:", round(y, 6), round(yp, 6))
105     print("difference:", abs(y - yp))
106
107     # -----
108     # 5. Receptive field: how far does information travel?
109     # -----
110     print("=" * 62)
111     Ahat = A + np.eye(4)
112     for k in (1, 2, 3):
113         reach = (np.linalg.matrix_power(Ahat, k) > 0).astype(int)
114         print(f"nodes reachable in <= {k} hops:\n", reach)

```

Listing 1: *mpnn_numpy.py*. A complete message-passing forward pass on formaldehyde, with a permutation test and a receptive-field calculation.

14.1 What to look at in the output

1. H^0 shows that the two hydrogens start with identical rows, since their input features are identical.
2. The aggregate for carbon is uniformly larger than for the other atoms, which is sum aggregation reporting degree.
3. After permutation the printed comparison returns `True` and the graph-level outputs agree to the last printed digit, with a difference of exactly 0. Invariance is a property of the architecture and holds before training.
4. The reachability matrices show the receptive field saturating after two hops, because formaldehyde has diameter two.

14.2 The same model in a production library

Written with PyTorch Geometric, the identical architecture becomes the following. The library supplies the scatter operations, the batching machinery and a large collection of published layers.

```

1  import torch
2  from torch.nn import Sequential, Linear, ReLU, GRU
3  from torch_geometric.nn import NNConv, global_add_pool
4  from torch_geometric.loader import DataLoader
5
6  class MPNN(torch.nn.Module):
7      def __init__(self, f_v, f_e, dim=64, layers=3):
8          super().__init__()
9          self.embed = Linear(f_v, dim)
10         edge_mlp = Sequential(Linear(f_e, 128), ReLU(), Linear(128, dim * dim))
11         self.conv = NNConv(dim, dim, edge_mlp, aggr="add") # sum aggregation
12         self.gru = GRU(dim, dim) # shared update
13         self.layers = layers
14         self.head = Sequential(Linear(dim, dim), ReLU(), Linear(dim, 1))
15
16     def forward(self, data):
17         h = torch.relu(self.embed(data.x))
18         m = h.unsqueeze(0)
19         for _ in range(self.layers): # weights shared across
20             msg = torch.relu(self.conv(h, data.edge_index, data.edge_attr))
21             h, m = self.gru(msg.unsqueeze(0), m)
22         h = h.squeeze(0)

```



```
23         g = global_add_pool(h, data.batch)           # sum readout: extensive
           target
24         return self.head(g).view(-1)
25
26 # ---- training loop -----
27 model = MPNN(f_v=23, f_e=6)
28 opt = torch.optim.Adam(model.parameters(), lr=1e-3)
29 loader = DataLoader(train_set, batch_size=64, shuffle=True)
30
31 for epoch in range(200):
32     model.train()
33     for batch in loader:           # batch is one disjoint-union graph
34         opt.zero_grad()
35         loss = torch.nn.functional.l1_loss(model(batch), batch.y)
36         loss.backward()
37         opt.step()
```

Listing 2: A trainable version. NNConv conditions the message weights on the bond features, which is the bond-conditioned variant of Section 8.

Three details worth noting in the listing.

Weights are shared across the message-passing layers, so depth costs iterations rather than parameters. The GRU acts as the update function and gives the atom state a memory of its own history, which counteracts oversmoothing. Readout is a sum, appropriate to an extensive target, and would be replaced by a mean for an intensive one.

14.3 Suggested laboratory sequence

1. Run the NumPy listing unchanged and confirm the permutation test.
2. Replace sum aggregation by mean and re-run the worked example. Explain what happens to the carbon aggregate and why.
3. Extend the molecule to acetaldehyde by adding a methyl carbon, and count how many layers are now required before the oxygen and the methyl hydrogens have communicated.
4. Build ethanol and dimethyl ether. Verify that the parameter-free model of Section 5 assigns them the same readout and that the message-passing model does not.
5. Move to the PyTorch listing on a public solubility set. Train once with a random split and once with a scaffold split, and compare the two reported errors.

15 Problems

Exercises 1 to 10 appear in the body of the text. Three further problems follow.

Exercise 11. A message-passing model with K layers, hidden width d , sum aggregation and dense ϕ is applied to a molecule with n atoms and m directed edges. Count the multiplications required by one forward pass, as a function of n , m , d and K , and identify which term dominates for a typical organic molecule.

Exercise 12. Design a featurisation for a heterogeneous catalyst surface with an adsorbate, given that the system is periodic and has no bond table. State the cutoff, the node features, the edge features and the readout, and justify each choice in one sentence.

Exercise 13. A colleague reports that adding a virtual node improved validation accuracy on a dataset of small molecules by a large margin. Propose two explanations, one architectural and one concerning the data, and describe an experiment that distinguishes them.

A Solutions

1. The adjacency matrix costs n^2 numbers and the edge list $2m$. For benzene with explicit hydrogens, 144 against 24. For the protein, 2.25×10^6 against 3060, a factor of about 735. With $m = 1.05n$ the edge list is cheaper whenever $n^2 > 2.1n$, that is for every graph with more than two nodes. The adjacency matrix is never the economical choice and survives only because it is convenient notation.

2. Invariant: molecular mass, dipole magnitude, any thermodynamic property. Equivariant: the vector of Mulliken charges, which permutes exactly as the atoms do. Neither: the index of the first atom in the input file, which is a property of the file and not of the molecule.

3. An integer code imposes a spurious metric, so a model would infer that element 6 and element 7 are more similar than element 6 and element 8, which is false in general and disastrous when the alphabet includes both first-row and heavy elements. The atomic number as a scalar is preferable when the target depends smoothly on it, for example core-level binding energies or heavy-atom scattering factors, and when the alphabet is large enough that one-hot encoding would be unmanageably sparse.

4. Writing $\tilde{A} = A + I$ gives $(\tilde{A}H)_i = h_i + \sum_{j \in \mathcal{N}(i)} h_j$, which is the update with $\mathcal{N}(i) \cup \{i\}$ in place of $\mathcal{N}(i)$. Omitting the self-loop discards the atom's own identity at every layer, so after one layer a carbon and a nitrogen with the same neighbours become indistinguishable. Element identity is lost.

5. For a d -regular graph every degree equals d , so $1/\sqrt{d_i d_j} = 1/d$ and the sum becomes a plain average scaled by $1/d$. Benzene without hydrogens is 2-regular, as is any cycloalkane skeleton. A branched structure such as neopentane is far from regular, with degrees ranging from one to four.

6. Take $\phi_{\text{msg}}(h_i, h_j, e_{ji}) = \text{MLP}([h_i \parallel h_j \parallel e_{ji}])$ and note that the concatenation is not symmetric in i and j , so the message from O to C already differs from the message from C to O whenever the two hidden states differ. The layer remains permutation equivariant, because the asymmetry is defined by the roles of sender and receiver and not by the node indices.

7. All atoms in both graphs begin with the same colour and have exactly two neighbours of that colour, so the refinement reaches its fixed point immediately and the colour multisets are $\{c\}^{12}$ in both cases. A node feature giving the size of the smallest ring containing the atom takes value six in the first graph and twelve in the second, and separates them at the input layer.

8. Mean pooling produces a size-independent output while the target grows roughly linearly with the number of heavy atoms. The signed error is therefore increasingly negative for large molecules and positive for small ones, crossing zero near the mean size of the training set, a signature that is unmistakable once plotted.

9. A scalar-only architecture predicts the energy and obtains forces as $F_i = -\partial E / \partial r_i$ by automatic differentiation, which is exactly equivariant by construction. The price is that every force evaluation requires a backward pass through the entire model, roughly doubling to tripling the cost of inference, and that the force accuracy is limited by the smoothness of the learned energy surface.

10. Against: 400 examples cannot constrain the tens of thousands of parameters in a five-layer model, and the seed-to-seed variance will exceed any architectural effect. For: with heavy regularisation, weight sharing across layers and a pretrained initialisation the model may still help. The insisted baseline is a gradient-boosted regressor on extended-connectivity fingerprints with the same split, reported over at least five seeds. If the graph model does not clear that baseline by more than the seed spread, it has not been demonstrated to work.

11. The embedding costs $nf_v d$. Each layer computes m messages at $O(d^2)$ each and n updates at $O(d^2)$ each, giving $O(K(m+n)d^2)$ overall. For an organic molecule $m \approx 2n$, so the message term dominates by a factor of about two and the whole cost is linear in molecular size, which is the central practical advantage of message passing over any n^2 formulation.

12. A defensible answer: cutoff of 5 to 6 Å with periodic images included, node features consisting of element one-hot, oxidation state where known and a flag distinguishing adsorbate from slab, edge features consisting of the interatomic distance expanded in a Gaussian radial basis together with a periodic-image flag, and readout restricted to the adsorbate atoms plus the surface layer if the target is an adsorption energy, since bulk atoms contribute a constant that only adds variance.

13. Architectural explanation: the property depends on interactions farther apart than the receptive field, so the virtual node supplies the missing long-range channel. Data explanation: the property correlates with molecular size or with a global count, which the virtual node reads off directly as a shortcut. The distinguishing experiment is to regress the target on molecular size alone. If the trivial regression explains most of the gain, the virtual node found a shortcut rather than chemistry.

B Glossary

Aggregator

A permutation-invariant reduction over a neighbourhood, typically sum, mean or maximum.

Equivariance

The property that transforming the input transforms the output in the same way.

Message passing

The scheme in which each node builds a message for each neighbour, aggregates the incoming messages and updates its own state.

Oversmoothing

Convergence of node representations to a common vector under repeated neighbourhood averaging.

Oversquashing

Loss of information when messages from an exponentially large neighbourhood are forced through a narrow structural bottleneck.

Readout

The final permutation-invariant reduction from node states to a graph-level prediction.

Receptive field

The set of nodes that can influence a given node, equal to the ball of radius K for a model with K layers.

Scaffold split

A data partition that assigns entire Bemis–Murcko frameworks to a single side, testing generalisation across chemical series.

Virtual node

An artificial node adjacent to all others, added to shorten the communication distance across the graph.

Weisfeiler–Leman test

A colour refinement procedure that bounds the ability of standard message passing to distinguish graphs.

C Annotated further reading

B. Sanchez-Lengeling, E. Reif, A. Pearce, A. B. Wiltschko, *Distill* 6, e33 (2021).

The source article for the present tutorial. Its interactive figures and its playground of trained models on an odour dataset are worth an hour of directed exploration and cannot be reproduced on paper.

A. Daigavane, B. Ravindran, G. Aggarwal, *Distill* 6, e32 (2021).

Companion article deriving graph convolution from the spectral side, and the right next reading for a student who wants the Laplacian picture.

J. Gilmer *et al.*, *Proc. ICML* 70, 1263 (2017).

The paper that unified the early architectures under the message-passing formalism used throughout Section 7.

P. W. Battaglia *et al.*, *arXiv:1806.01261* (2018).

The graph-attribute formalism of Figure 6, and an extended argument for relational inductive bias.

K. Xu, W. Hu, J. Leskovec, S. Jegelka, *Proc. ICLR* (2019).

The expressivity result of Section 10 and the GIN architecture that attains the bound.

K. T. Schütt *et al.*, *J. Chem. Phys.* 148, 241722 (2018).

SchNet, the standard entry point to distance-based models for chemistry.

I. Batatia *et al.*, *Proc. NeurIPS* 35 (2022).

MACE, representative of the equivariant potentials at the top of the ladder in Figure 9.

End of tutorial